# Sri Lanka Institute of Information Technology




## IT3091 - Machine Learning

## GROUP ASSIGNMENT – FINAL REPORT

**Group Number: 2026-DS-35**

| Student ID | Student Name |
|---|---|
| IT24104087 | Rathnamalala R.I.B.T |
| IT24103907 | Pamindi M.M. |
| IT24400066 | Athauda P.A.K.D. |
| IT24103888 | Gunathunga K.S.N.T. |

## Table of Contents


1. Introduction & Business Problem ..... 3
2. Data Understanding ..... 3
3. Data Dictionary ..... 5
4. Exploratory Data Analysis ..... 8
5. Data Preprocessing ..... 11
6. Feature Engineering & Selection ..... 11
7. Modeling Strategy ..... 12
8. Model Development & Tuning ..... 12
9. Model Evaluation & Comparison ..... 15
10. Final Model Selection ..... 17
11. Business Interpretation & Recommendations ..... 17
12. Limitations & Responsible AI ..... 18
13. Conclusion ..... 18
14. Appendix 01: Decision Log ..... 19
15. Appendix 02: Youtube Video Link ..... 20

## 1. Introduction & Business Problem

Supply-chain operations depend on reliable delivery. Late orders can reduce customer satisfaction, increase complaint-handling work and create avoidable operational costs. This project develops a binary classification approach to flag orders that are at risk of late delivery, using the DataCo Supply Chain dataset.

The unit of analysis is one order/shipment record. The target is Late_delivery_risk, where 1 indicates late delivery and 0 indicates no late-delivery risk. The intended user is the logistics or operations planning team, which could use a risk score to prioritise checks and interventions before dispatch. Delay patterns by shipping mode, market and product category are also examined to help explain the model's predictions; these analyses are explanatory rather than separate prediction targets.

## 2. Data Understanding

The supplied dataset description reports 180,519 records and 53 original columns. The target distribution is 54.83% late and 45.17% not late, so the classes are relatively balanced. Available fields include order and customer geography, shipping mode, order dates, product/category information, quantities, sales, discounts and profit-related measures.

Data quality checks found no fully duplicated rows. Product Description was entirely missing and Order Zipcode was approximately 86% missing. Customer Lname and Customer Zipcode had only a small number of missing values, but those fields were not retained because they were personally identifying or unsuitable for generalisable prediction. The modelling workflow drops fields that disclose the outcome or are only known after delivery, especially Delivery Status, Days for shipping (real) and shipping date (DateOrders).

The data dictionary was used to classify the original fields by meaning, type, missingness and modelling disposition. Key retained fields include Late_delivery_risk (binary target), Days for shipment (scheduled), Shipping Mode, Order Status, Market, Order Region, Customer Segment, Category Name, order-item sales/discount/profit measures and geographic context. The full dataset contains 53 original columns; the final feature matrix contains 32 features after engineering and encoding.

| Field / field group | Type | Use / decision |
|---|---|---|
| Late_delivery_risk | Binary integer (0/1) | Target: 1 = late-delivery risk; 0 = not late. |

| | | |
|---|---|---|
| Days for shipment (scheduled) | Integer | Kept: promised delivery window; available before shipment completion. |
| Shipping Mode; Order Status | Categorical text | Kept: operational context; Order Status retained because cancellation/fraud status is a real observed pattern. |
| Market; Order Region; Order Country; Customer Country/State | Categorical text | Selected geography fields kept; redundant/high-cardinality city/state fields removed as recorded in the decision log. |
| Sales, Order Item Total, Order Item Quantity, Order Item Discount/Rate, Order Item Profit Ratio, Benefit per order, Order Profit Per Order, Product Price | Numeric | Kept as commercial/order context features. |
| Customer Segment, Department Name/Id, Category Name/Id | Categorical / coded values | Kept to represent customer and product context. |
| DateOrders order date | Datetime/text | Used only to derive order_dow, order_month, order_day, order_hour and order_is_weekend; raw date then dropped. |
| Delivery Status; Days for shipping (real); shipping date (DateOrders) | Text / numeric / datetime | Dropped to prevent target leakage because these fields reveal or depend on the completed delivery outcome. |
| Customer names, email, password, street; Product Image/Description | Text / URL / empty | Dropped to reduce privacy exposure, noise and unusable free text; Product Description was 100% missing. |
| Order Zipcode; Customer Zipcode; Product Status | Numeric / constant | Dropped: Order Zipcode had about 86% missing; Customer Zipcode was redundant; Product Status had no variance. |

| | | |
|---|---|---|
| Order/customer/item/product IDs; Customer City; Order City/State; Product Name | Identifiers / high-cardinality text | Dropped as identifiers or redundant high-cardinality fields with limited generalisable value. |

## 3. Data Dictionary

| Column | Type | Working definition | Notebook treatment |
|---|---|---|---|
| Type | object | Payment transaction type | Keep; one-hot encode |
| Days for shipping (real) | int64 | Actual shipping duration in days | Drop: post-outcome leakage |
| Days for shipment (scheduled) | int64 | Scheduled shipping duration in days | Keep numeric |
| Benefit per order | float64 | Profit/benefit generated by an order | Keep numeric |
| Sales per customer | float64 | Sales value associated with the customer/order | Keep numeric |
| Delivery Status | object | Observed delivery outcome/status | Drop: directly reveals target |
| Late_delivery_risk | int64 | Binary target: 1 = late, 0 = not late | Target |
| Category Id | int64 | Category identifier | Drop: ID |
| Category Name | object | Product category name | Keep; smoothed target encoding |
| Customer City | object | Customer city | Keep; smoothed target encoding |
| Customer Country | object | Customer country | Keep; low-cardinality categorical |
| Customer Email | object | Customer email address | Drop: PII |
| Customer Fname | object | Customer first name | Drop: PII |
| Customer Id | int64 | Customer identifier | Drop: ID |
| Customer Lname | object | Customer last name | Drop: PII |
| Customer Password | object | Customer password field | Drop: sensitive PII |
| Customer Segment | object | Customer segment/category | Keep; one-hot encode |

| | | | |
|---|---|---|---|
| Customer State | object | Customer state | Keep; smoothed target encoding |
| Customer Street | object | Customer street address | Drop: PII |
| Customer Zipcode | float64 | Customer postal/ZIP code | Drop: PII; a few missing |
| Department Id | int64 | Department identifier | Drop: ID |
| Department Name | object | Department name | Keep; one-hot encode |
| Latitude | float64 | Geographic latitude | Keep numeric |
| Longitude | float64 | Geographic longitude | Keep numeric |
| Market | object | Sales market/region group | Keep; one-hot encode |
| Order City | object | Order destination city | Keep; smoothed target encoding |
| Order Country | object | Order destination country | Keep; smoothed target encoding |
| Order Customer Id | int64 | Customer identifier attached to order | Drop: ID |
| order date (DateOrders) | object | Order timestamp | Convert to datetime; derive month/day-of-week |
| Order Id | int64 | Order identifier | Drop: ID |
| Order Item Cardprod Id | int64 | Order-item product/card identifier | Drop: ID |
| Order Item Discount | float64 | Discount amount on order item | Keep numeric |
| Order Item Discount Rate | float64 | Discount rate on order item | Keep numeric |
| Order Item Id | int64 | Order-item identifier | Drop: ID |
| Order Item Product Price | float64 | Unit product price recorded for order item | Keep numeric |
| Order Item Profit Ratio | float64 | Profit ratio for order item | Keep numeric |
| Order Item Quantity | int64 | Quantity ordered | Keep numeric |
| Sales | float64 | Gross sales amount | Keep numeric |
| Order Item Total | float64 | Net/total value of order item | Keep numeric |

| | | | |
|---|---|---|---|
| Order Profit Per Order | float64 | Profit per order | Keep numeric |
| Order Region | object | Order region | Keep; smoothed target encoding |
| Order State | object | Order destination state | Keep; smoothed target encoding |
| Order Status | object | Order processing/status category | Keep; one-hot encode |
| Order Zipcode | float64 | Order postal/ZIP code | Drop: ~86% missing |
| Product Card Id | int64 | Product/card identifier | Drop: ID |
| Product Category Id | int64 | Product category identifier | Drop: ID |
| Product Description | float64 | Product description field | Drop: 100% missing |
| Product Image | object | Product image URL/path | Drop: free-text/non-predictive |
| Product Name | object | Product name | Keep; smoothed target encoding |
| Product Price | float64 | Listed product price | Keep numeric |
| Product Status | int64 | Product status code | Drop: ID/constant in notebook |
| shipping date (DateOrders) | object | Shipment timestamp | Drop: post-order leakage |
| Shipping Mode | object | Selected shipping service/mode | Keep; one-hot encode |

## 4. Exploratory Data Analysis

EDA was used to check the target distribution, inspect delay rates across operational categories and understand relationships among numeric features. Shipping Mode and Days for shipment (scheduled) are closely related because each shipping mode corresponds to a scheduled shipping duration. They are strong signals in this dataset. Market, order region and product category also show differences in late-delivery rates, while most individual sales/profit measures have weaker relationships with the target on their own.

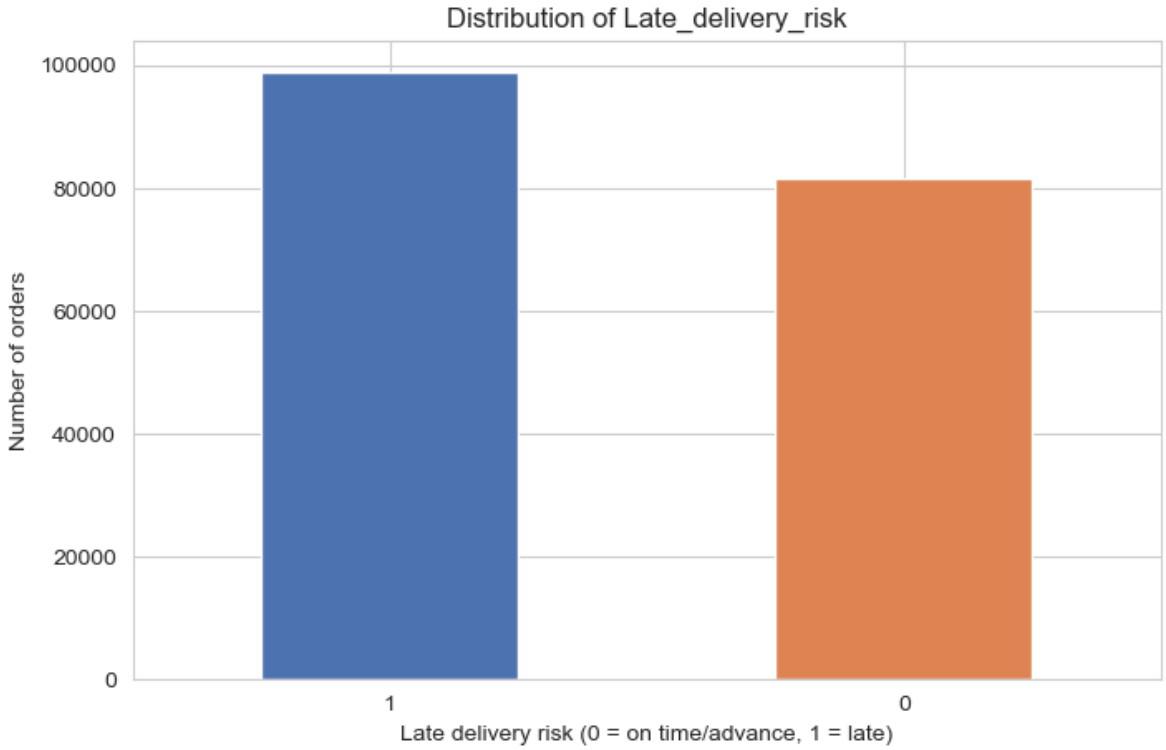



*Figure 1. Distribution of the Late_delivery_risk target*




*Figure 2. Late-delivery rate by shipping mode*

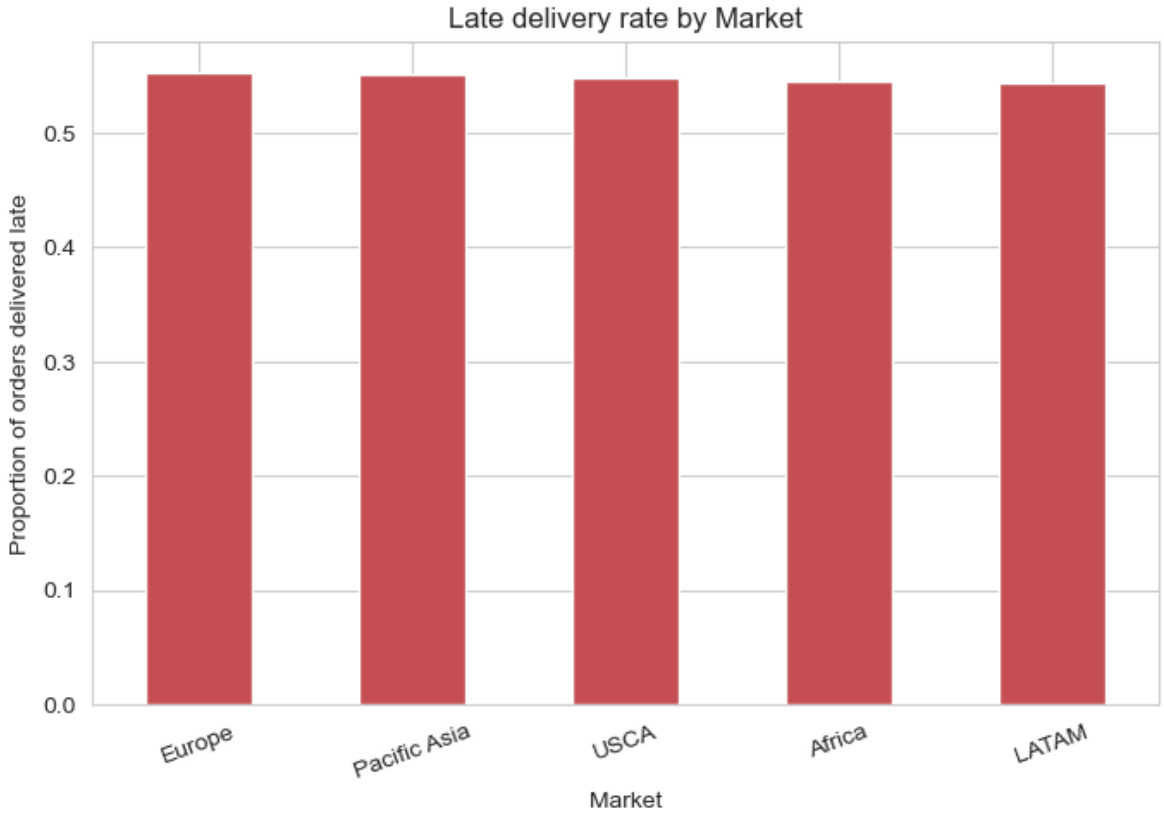



*Figure 3. Late-delivery rate by market*

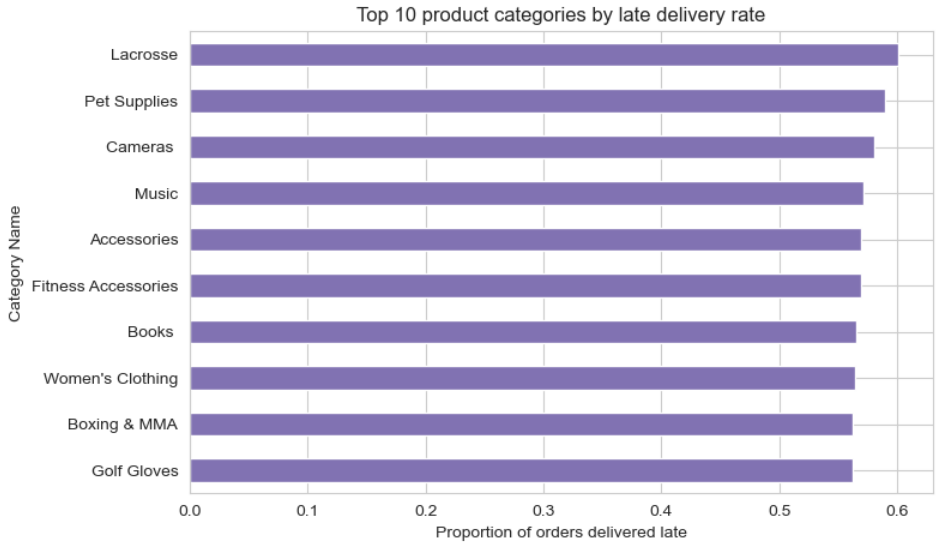



*Figure 4. Top product categories by late-delivery rate (categories with sufficient orders)*

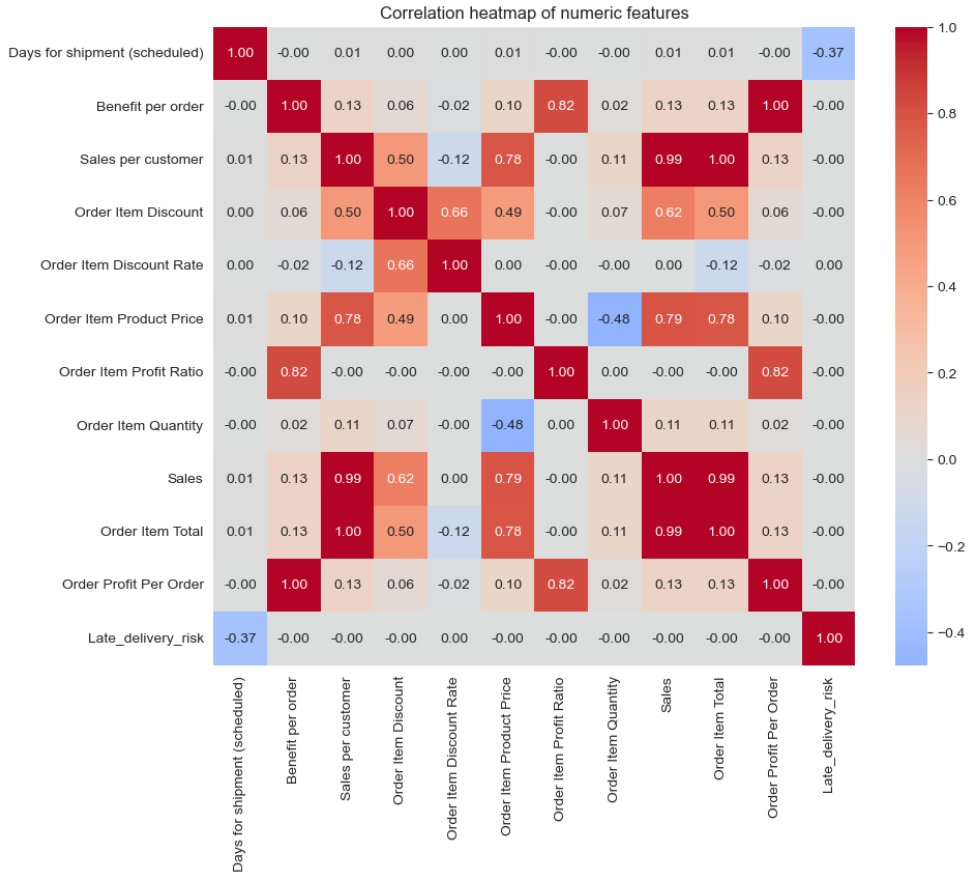



*Figure 5. Correlation heatmap for selected numeric features*

These plots are descriptive, not proof that a feature causes delays. Shipping mode and scheduled duration can help prioritise orders, but operational teams should investigate process and capacity factors before making changes based only on observed associations.

## 5. Data Preprocessing

Preprocessing decisions were recorded in the project decision log and applied to a copy of the source dataframe so the original remained unchanged. Delivery Status, Days for shipping (real) and shipping date (DateOrders) were removed as leakage. Customer email, password, names, street address, product image and free-text/empty product description were removed for privacy and data-quality reasons. Order/customer/item/product IDs were removed because they identify records rather than generalisable patterns. Customer City, Order City, Order State and Product Name were removed because of high cardinality or overlap with broader geography/category fields. Order Zipcode (about 86% missing), redundant Customer Zipcode and constant Product Status were also dropped. No exact duplicate rows were found; negative profit values were retained because they can represent genuine business outcomes.

The order date was converted to datetime and decomposed into order_dow, order_month, order_day, order_hour and order_is_weekend; the raw date was then dropped. The decision log records that no distance feature was engineered from latitude/longitude because Market and Order Region already summarised geography at a useful level. Remaining retained columns had no missing values, so imputation was unnecessary. The target was close to balanced (about 55% late and 45% not late), so we did not apply SMOTE or other resampling.

## 6. Feature Engineering & Selection

Date features order_month and order_dayofweek were engineered to represent possible seasonal and weekly patterns. High-cardinality fields—including city, product/category and geographic fields—were smoothed target-encoded to avoid creating thousands of one-hot columns. The encoding maps were learned from the training set only; unseen categories fall back to the overall training-set late-delivery rate. The smoothing value was set to k=20.

Categorical encoding followed feature creation. The decision log documents label encoding for the final tree-focused pipeline because tree models split on individual encoded values and several fields were high-cardinality (for example, Order Country). Lower-cardinality nominal variables were considered for one-hot encoding in the alternate report-aligned workflow; the final submitted notebook should be treated as the source of truth for the exact encoding implementation. Numeric business measures were retained as numeric inputs.

## 7. Modeling Strategy

Four supervised classification methods were compared to balance interpretability and predictive performance: Logistic Regression as the baseline, Decision Tree as an interpretable rule-based model, Random Forest as a bagged tree ensemble, and XGBoost as a boosted tree ensemble. Each model predicts the same binary target using the leakage-controlled feature set.

The approach compares a transparent linear baseline against tree methods that can represent non-linear relationships and feature interactions. Accuracy alone was not treated as sufficient. Evaluation includes precision, recall, F1 score and ROC AUC, with stratified cross-validation used as an additional robustness check.

## 8. Model Development & Tuning

The decision log specifies a reproducible 80/20 stratified train/test split with random_state=42, using the same split for all four methods. Logistic Regression served as the baseline, with StandardScaler applied and regularisation strength compared across C values from 0.01 to 10. Decision Tree depth was compared across several values to balance accuracy and overfitting. Random Forest settings were checked using n_estimators (100, 200) and max_depth (10, unlimited). XGBoost learning rates (0.05, 0.1, 0.2) were compared, with 0.1 selected as a balance of performance and training time. Train/test accuracy gaps were inspected, and stratified cross-validation was used as a robustness check.

Key decision is no resampling was used because the target was not severely imbalanced, leakage and privacy fields were excluded before splitting/model fitting, identical stratified splits support fair model comparison and model selection considered precision, recall, F1 and ROC AUC rather than accuracy alone.

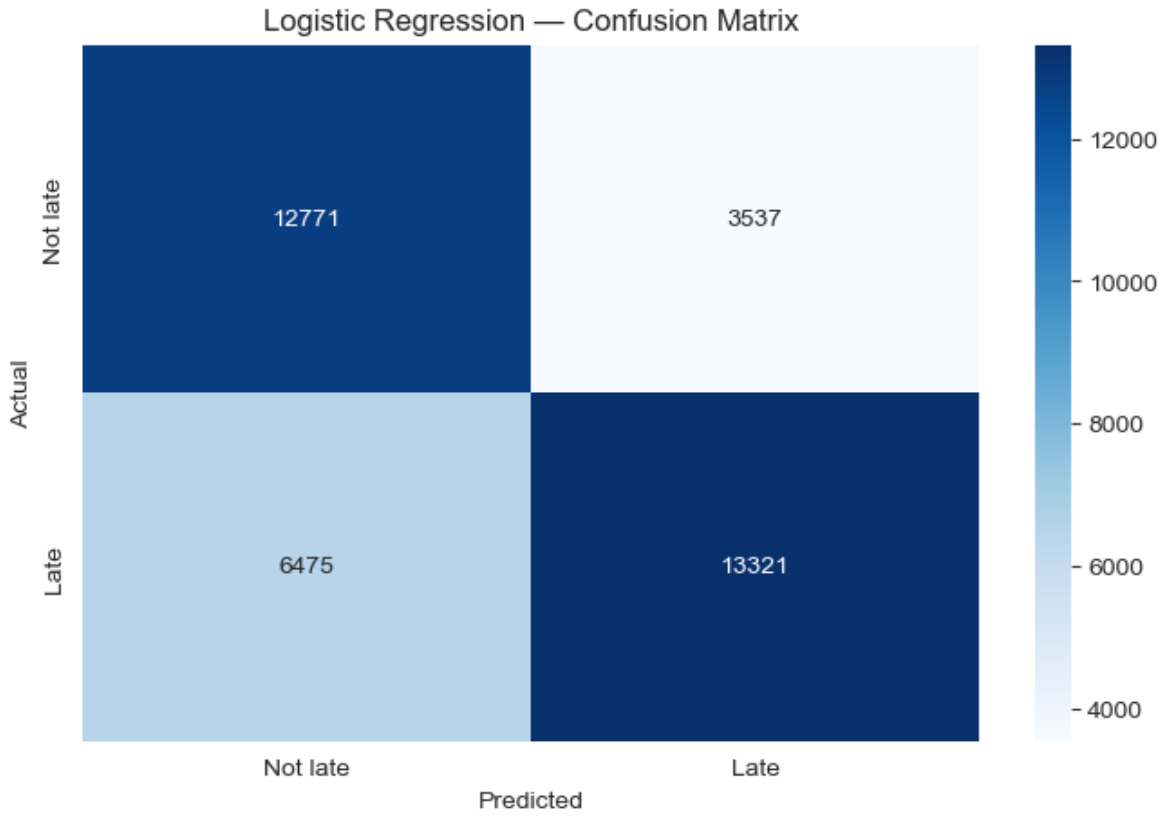



*Figure 6. Logistic Regression confusion matrix*

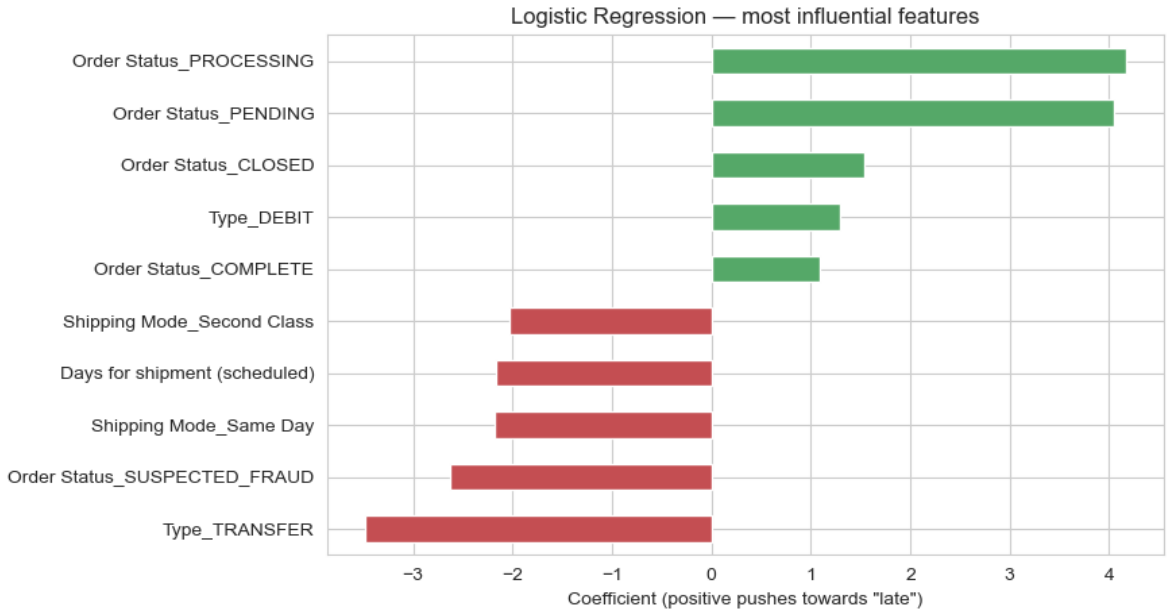



*Figure 7. Largest positive and negative Logistic Regression coefficients*

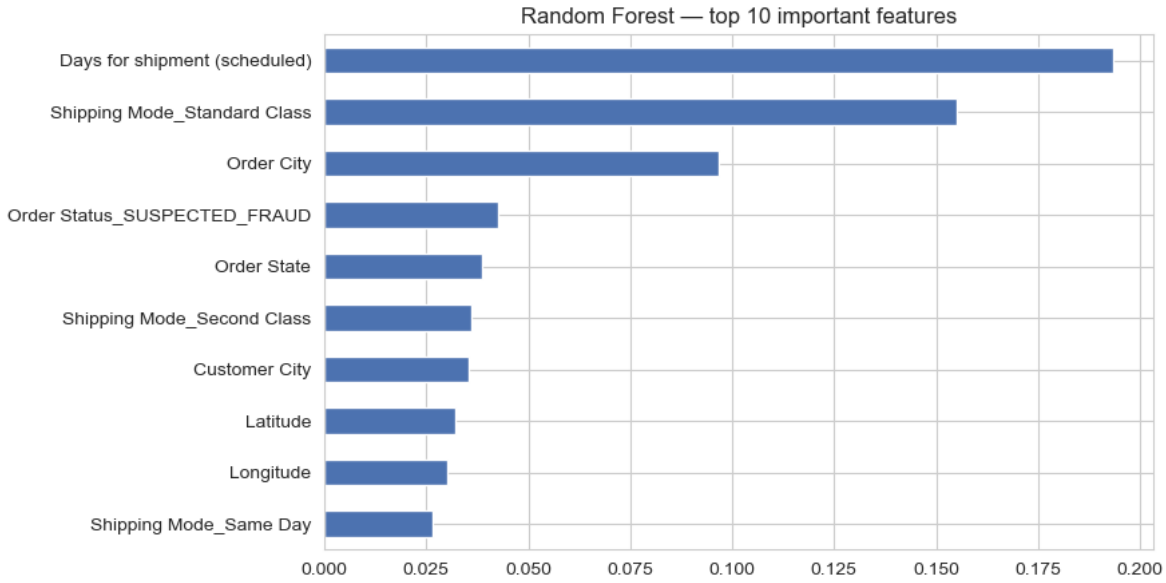



*Figure 8. Random Forest feature importance*

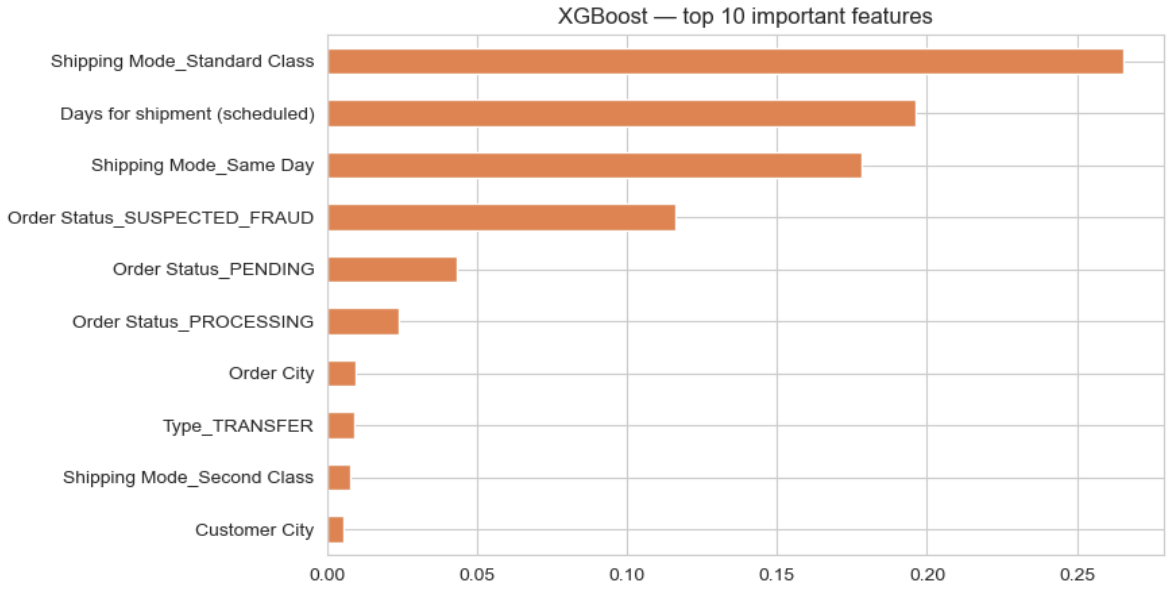



*Figure 9. XGBoost feature importance*

The confusion matrix shows the baseline's types of classification errors, while coefficient and feature-importance plots provide model-specific interpretability. Feature importance is not causal evidence and can be influenced by correlated features and the way each model represents the data.

## 9. Model Evaluation & Comparison

| Model | Method | Accuracy | Precision | Recall | F1 | ROC AUC | CV Accuracy | Interpretation |
|---|---|---|---|---|---|---|---|---|
| Logistic Regression | Baseline linear classifier | 0.7227 | 0.7902 | 0.6729 | 0.7269 | 0.8028 | 0.7378 ± 0.0022 | Simple and interpretable, but weaker on non-linear interactions. |
| Decision Tree | Single rule-based tree | 0.7644 | 0.8209 | 0.7295 | 0.7725 | 0.8485 | 0.7043 ± 0.0048 | Strong test result and easy to explain; single trees can be less stable. |
| Random Forest | Bagged tree ensemble | 0.7601 | 0.8436 | 0.6904 | 0.7594 | 0.8540 | 0.7355 ± 0.0032 | Highest precision; robust ensemble and useful backup option. |
| XGBoost | Boosted tree ensemble | 0.7742 | 0.8347 | 0.7335 | 0.7808 | 0.8681 | 0.7401 ± 0.0026 | Best test Accuracy, F1 and ROC AUC; notebook's recommended operational model. |

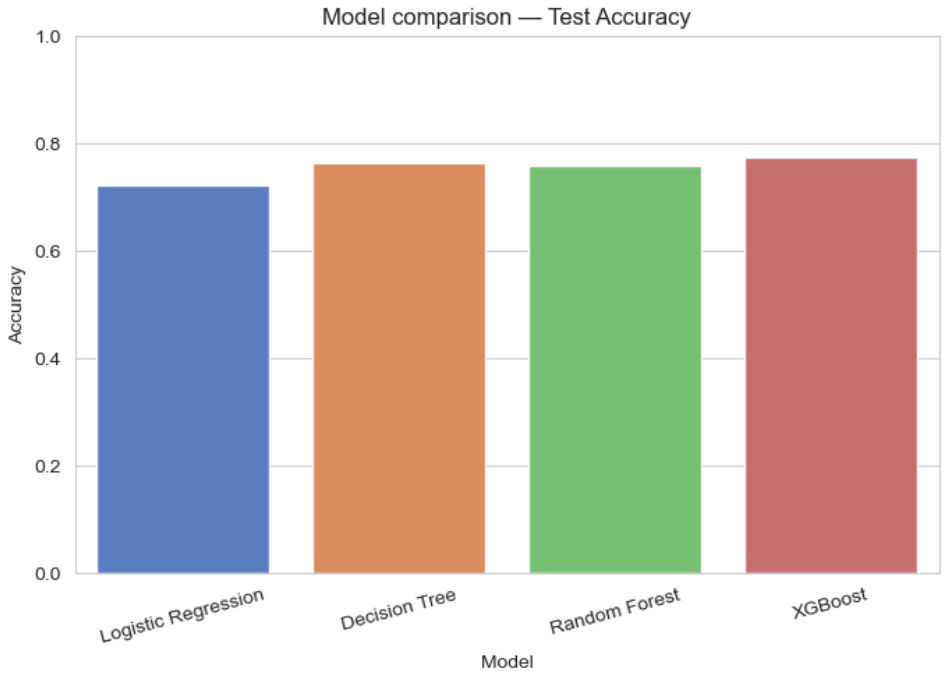



*Figure 10. Held-out test accuracy across the four models*

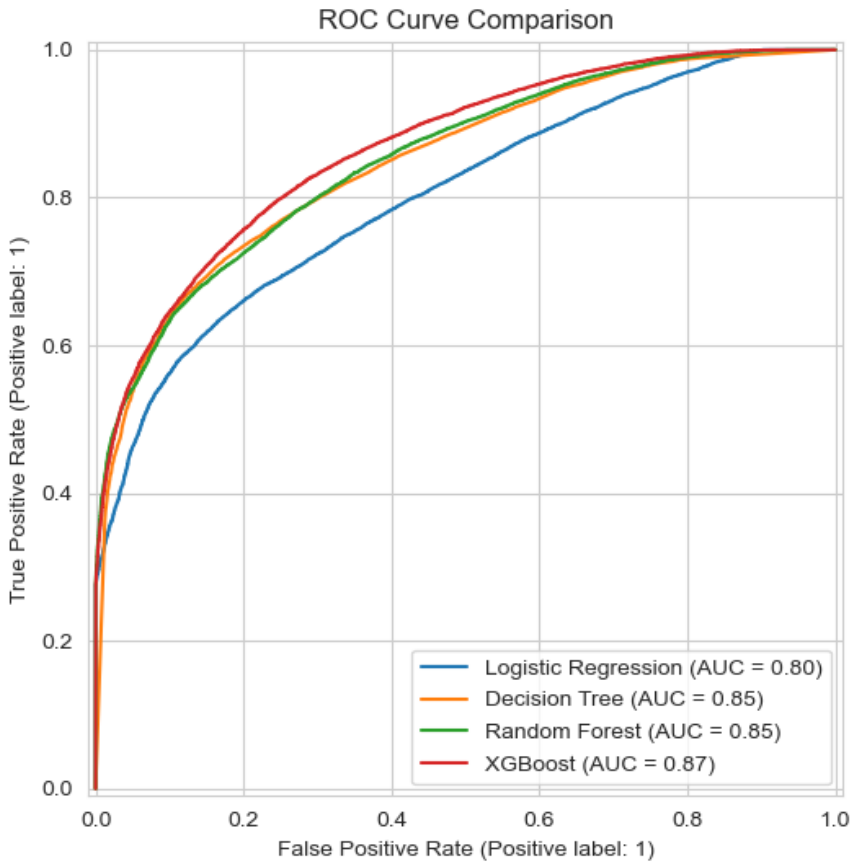



*Figure 11. ROC curve comparison for all four models*

Logistic Regression provides a transparent baseline but has the lowest test accuracy and ROC AUC, indicating that a purely linear decision boundary does not capture all delay-risk interactions.

Decision Tree improves substantially over the baseline and is useful for explaining human-readable rules. Its cross-validation mean is lower than its held-out test accuracy, so it is best treated as an interpretable supporting model rather than the main deployment choice.

Random Forest produces the highest precision (0.8436), meaning that among orders it flags as late, a high proportion are actually late. Its recall is lower than the Decision Tree and XGBoost.

XGBoost has the strongest overall held-out performance: Accuracy 0.7742, F1 0.7808 and ROC AUC 0.8681. The notebook therefore recommends XGBoost as the primary model, with Random Forest as a backup and the Decision Tree as an explainability aid.

The cross-validation results are close with low standard deviations, supporting reasonably stable model behaviour across folds. The CV ordering differs somewhat from the held-out test ordering, so the recommendation is based on the full set of test metrics rather than accuracy alone.

## 10. Final Model Selection

XGBoost is selected as the primary candidate because it gives the strongest overall held-out performance across accuracy, F1 and ROC AUC. Random Forest is retained as a backup candidate due to its high precision and ensemble robustness. A Decision Tree can support explanation of simple decision rules for non-technical stakeholders. This is a model recommendation based on the available historical dataset, not evidence that a live deployment has been completed.

## 11. Business Interpretation & Recommendations

The selected XGBoost model can be used to estimate the risk of late delivery for orders in the DataCo supply-chain dataset. In a practical setting, this risk score could help supply-chain staff identify orders that may require additional attention before dispatch. By reviewing the predicted risk alongside shipping mode, scheduled shipping duration, market, and product and order details, staff could investigate possible delays and take preventive action where appropriate. The model should support operational decisions rather than make automatic decisions about customers or shipments.

- **Pilot the XGBoost model:** Test the model on new or previously unseen orders and display its predicted late-delivery risk in a simple monitoring dashboard for operations staff. Treat this as a proposed pilot, not an implemented system.
- **Prioritise high-risk orders:** Use a risk threshold to create a list of orders for early review. Staff could check dispatch readiness, shipping arrangements, carrier availability and other relevant operational information before deciding whether action is needed.

- **Choose a suitable alert threshold:** Evaluate different classification thresholds using precision and recall. Select a threshold with operations stakeholders that balances missed late deliveries against the workload caused by unnecessary alerts.
- **Compare model performance:** Use the evaluation results to compare XGBoost with Random Forest and the other tested models. Consider relevant metrics such as precision, recall, F1-score and ROC-AUC rather than relying on accuracy alone.
- **Monitor and review performance:** If the model is piloted, record the number of flagged orders, how many subsequently arrive late, and whether interventions appear useful. Re-evaluate the model when delivery patterns, shipping arrangements or the data available to the model change.
- **Keep human oversight:** Present the prediction as a risk estimate, not a guarantee that an order will be late. Allow operations staff to consider additional information and document their reasons for taking or not taking action.

## 12. Limitations & Responsible AI

The dataset reflects a fixed historical period, so model performance may decline when new cities or products appear, carriers change, or service levels are updated. Target encoding for unseen categories falls back to the overall training-set risk, which may limit prediction accuracy for unfamiliar categories. The model predicts the risk of late delivery rather than a guaranteed outcome, and it has not been validated through a prospective live trial. Geographic and product/order features may reflect existing operational differences. The absence of explicit demographic fields does not guarantee that the model is unbiased. Before deployment, the model should be tested across relevant operational groups, data access should be protected, and model performance should be monitored for changes over time. The alert threshold should reflect business costs, and a human decision-maker should remain responsible for operational actions.

## 13. Conclusion

This project established a leakage-aware pipeline for predicting late-delivery risk from order-level supply-chain data. EDA highlighted shipping mode/scheduled duration and differences across markets and categories. After removing leakage, personal information and unsuitable identifiers, the team engineered date features and encoded categorical variables, then compared four classifiers. XGBoost achieved the strongest overall test performance and is recommended as the primary candidate, with Random Forest as a high-precision alternative and Decision Tree as an explanation aid. Any operational use should remain human-supervised and include monitoring, fairness checks and revalidation as the supply-chain environment changes.

## 14.Appendix 01: Decision Log

| Decision Area | Decision | Reason |
|---|---|---|
| Business objective | Predict late-delivery risk before dispatch | Supports proactive operational intervention. |
| Unit of analysis | One order/shipment record | Matches the notebook's decision framing. |
| Target | Late_delivery_risk | Already supplied as a binary 0/1 label. |
| Secondary lens | Delay pattern analysis | Used to explain patterns by shipping mode, market and category; not a second prediction target. |
| Class imbalance | No SMOTE/class rebalancing | Target is relatively balanced: 54.83% late vs 45.17% not late. |
| Leakage control | Drop Delivery Status | It perfectly matches/reveals the target. |
| Leakage control | Drop Days for shipping (real) | Actual shipping time is only known after shipment and is used to determine lateness. |
| Leakage control | Drop shipping date (DateOrders) | Post-order information closely tied to delivery outcome. |
| Privacy | Drop names, email, password, street and ZIP fields | Avoid PII/sensitive fields and non-generalizable information. |
| Identifiers | Drop order/customer/product/category/department ID fields and Product Status | Identifiers do not represent stable predictive patterns. |
| Missing data | Drop Product Description and Order Zipcode | 100% missing and ~86% missing respectively. |
| Date handling | Extract order_month and order_dayofweek | Preserves possible seasonal/weekly patterns without using raw timestamp. |
| Split strategy | 80/20 stratified train-test split; random_state=42 | Preserves class proportions and reproducibility. |
| High-cardinality encoding | Smoothed target encoding, k=20 | Avoids thousands of one-hot columns; rare categories shrink toward overall rate. |

| | | |
|---|---|---|
| Leakage-safe encoding | Learn target-encoding maps from training set only | Prevents test information from leaking into training. |
| Unknown categories | Fallback to overall training late-delivery rate | Allows unseen test categories to be scored safely. |
| Low-cardinality encoding | One-hot encoding with drop_first=True | Appropriate for nominal categories with manageable cardinality. |
| Scaling | StandardScaler only for Logistic Regression | Linear coefficient model benefits from scaling; tree models do not require it. |
| Baseline model | Logistic Regression | Simple, fast and interpretable reference point. |
| Alternative models | Decision Tree, Random Forest, XGBoost | Tests non-linear rules, ensembles and boosting on tabular data. |
| Decision Tree controls | max_depth=16, min_samples_leaf=8 | Limits memorization/overfitting while retaining interpretable rules. |
| Random Forest controls | 300 trees, max_depth=18, min_samples_leaf=3 | Balances predictive power and overfitting control. |
| XGBoost controls | 350 trees, depth=7, learning_rate=0.1, subsample=0.9, colsample_bytree=0.9 | Strong tabular model with sampling regularization. |
| Evaluation metrics | Accuracy, Precision, Recall, F1, ROC AUC | Gives both threshold-based and ranking-quality views. |
| Robustness check | 3-fold stratified CV on 40,000-row training sample | Checks stability while keeping runtime reasonable. |
| Operational recommendation | XGBoost primary; Random Forest backup; Decision Tree for explanation | XGBoost has best test Accuracy, F1 and ROC AUC in the notebook. |

## 15.Appendix 02: Youtube Video Link

https://youtu.be/rt18aLY4P88